Algorithms

Lecture # 25

Syed Hamed Raza

Huffman Encoding: Correctness

Huffman Encoding: Correctness

Note that the binary tree constructed by Huffman algorithm is a full binary tree. Claim

- Consider two characters x and y with the smallest probabilities.
- Then there is optimal code tree in which these two characters are siblings at the maximum depth in the tree.

Huffman Encoding: Correctness

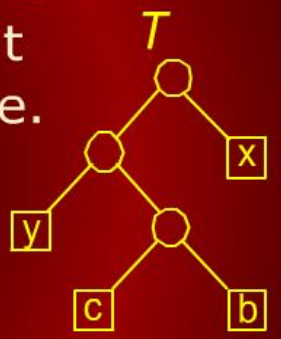
Huffman Encoding: Correctness

- Let T be any optimal prefix code tree with two siblings b and c at the maximum depth of the tree.
- Assume without loss of generality that

$$p(b) \leq p(c)$$

and

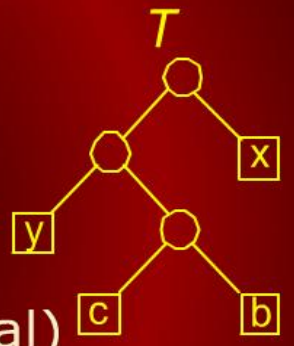
$$p(x) \leq p(y)$$



Huffman Encoding: Correctness

Huffman Encoding: Correctness

- Since x and y have the two smallest probabilities (we claimed this), it follows that



$p(x) \leq p(b)$ (may be equal)
and
$p(y) \leq p(c)$ (may be equal)

Huffman Encoding: Correctness

Huffman Encoding: Correctness

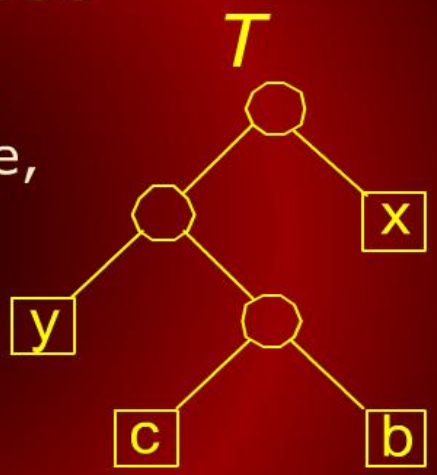
- Since b and c are at the deepest level of the tree, we know that

$$d(b) \geq d(x)$$

and

$$d(c) \geq d(y)$$

(d is the depth).



Huffman Encoding: Correctness

Huffman Encoding: Correctness

- Thus we have

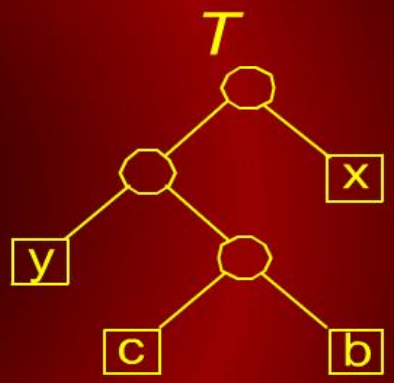
$$p(b) - p(x) \geq 0$$

and

$$d(b) - d(x) \geq 0$$

- Hence their product is non-negative. That is,

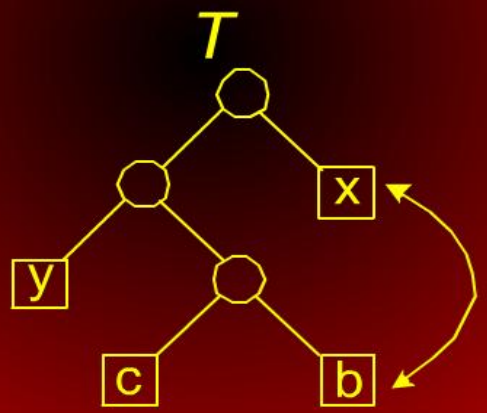
$$(p(b) - p(x)) \cdot (d(b) - d(x)) \geq 0$$



Huffman Encoding: Correctness

Huffman Encoding: Correctness

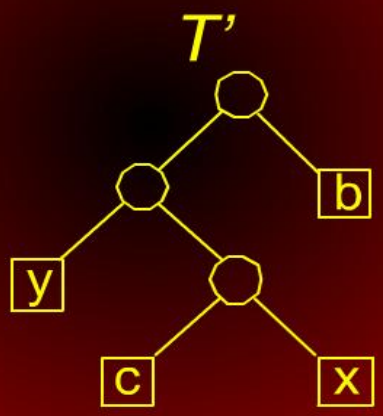
Now swap the positions of x and b in the tree



Huffman Encoding: Correctness

Huffman Encoding: Correctness

This results in a new tree T′



Let's see how the cost changes.

Huffman Encoding: Correctness

Huffman Encoding: Correctness

The cost of T' is

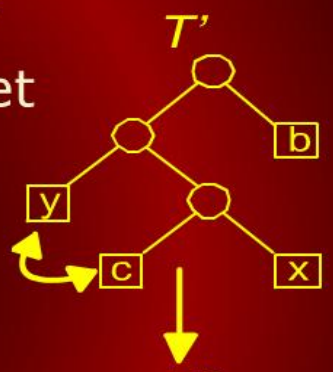
$$
\begin{aligned}
B(T') &= B(T) - p(x)d(x) + p(x)d(b) \\
&\quad - p(b)d(b) + p(b)d(x) \\
&= B(T) + p(x)[d(b) - d(x)] \\
&\quad - p(b)[d(b) - d(x)] \\
&= B(T) - (p(b) - p(x))(d(b) - d(x)) \\
&\leq B(T) \text{ because } (p(b) - p(x)) \\
&\quad (d(b) - d(x)) \geq 0
\end{aligned}
$$

Thus the cost does not increase, implying that T' is an optimal tree.

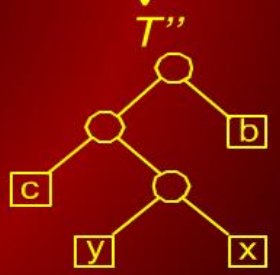
Huffman Encoding: Correctness

Huffman Encoding: Correctness

- By switching y with c we get the tree T″.



- Using a similar argument, we can show that T″ is also optimal.

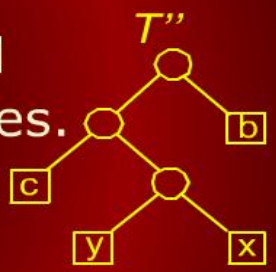


Huffman Encoding: Correctness

Huffman Encoding: Correctness

The final tree T'' satisfies the statement of the claim:

- Consider two characters x and y with the smallest probabilities.
- Then there is optimal code tree in which these two characters are siblings at the maximum depth in the tree.



Huffman Encoding: Correctness

Huffman Encoding: Correctness

- The claim we just proved asserts that the first step of Huffman algorithm is the proper one to perform (the greedy step).
- The complete proof of correctness for Huffman algorithm follows by induction on n.

Huffman Encoding: Correctness

Huffman Encoding: Correctness

Claim: Huffman algorithm produces the optimal prefix code tree.

Proof:

- The proof is by induction on n, the number of characters.
- For the basis case, n = 1, the tree consists of a single leaf node, which is obviously optimal.
- We want to show it is true with exactly n characters.

Huffman Encoding: Correctness

Huffman Encoding: Correctness

- Suppose we have exactly n characters.
- The previous claim states that two characters x and y with the lowest probability will be siblings at the lowest level of the tree.
- Remove x and y and replace them with a new character z whose probability is $p(z) = p(x) + p(y)$.
- Thus n - 1 characters remain.

Huffman Encoding: Correctness

Huffman Encoding: Correctness

- Consider any prefix code tree T made with this new set of n – 1 characters.
- We can convert T into prefix code tree T′ for the original set of n characters by replacing z with nodes x and y.
- This is essentially undoing the operation where x and y were removed an replaced by z.

Huffman Encoding: Correctness

Huffman Encoding: Correctness

The cost of the new tree T' is

$$
\begin{aligned}
B(T') &= B(T) - p(z)d(z) + p(x)[d(z) + 1] \\
&\quad + p(y)[d(z) + 1] \\
&= B(T) - (p(x) + p(y))d(z) + (p(x) \\
&\quad + p(y))[d(z) + 1] \\
&= B(T) + (p(x) + p(y)) \\
&\quad [d(z) + 1 - d(z)] \\
&= B(T) + p(x) + p(y)
\end{aligned}
$$

Huffman Encoding: Correctness

Huffman Encoding: Correctness

$$B(T') = B(T) + p(x) + p(y)$$

- The cost changes but the change depends in no way on the structure of the tree T (T is for n - 1 characters).
- Therefore, to minimize the cost of the final tree T′, we need to build the tree T on n - 1 characters optimally.
- By induction, this is exactly what Huffman algorithm does. Thus the final tree is optimal.

Activity Selection

Greedy Algorithms: Activity Selection

Greedy Algorithms: Activity Selection

- The activity scheduling is a simple scheduling problem for which the greedy algorithm approach provides an optimal solution.
- We are given a set $S = \{a_1, a_2, \ldots, a_n\}$ of n activities that are to be scheduled to use some resource.
- Each activity a_i must be started at a given start time s_i and ends at a given finish time f_i.

Greedy Algorithms: Activity Selection

Greedy Algorithms: Activity Selection

- An example is that a number of lectures are to be given in a single lecture hall.
- The start and end times have be set up in advance.
- The lectures are to be scheduled.

Greedy Algorithms: Activity Selection

Greedy Algorithms: Activity Selection

- There is only one resource (e.g., lecture hall)
- Some start and finish times may overlap.
- Therefore, not all requests can be honored.

Greedy Algorithms: Activity Selection

Greedy Algorithms: Activity Selection

- We say that two activities a_i and a_j are non-interfering if their start-finish intervals do not overlap.
- i .e, $(s_i, f_i) \cap (s_j, f_j) = \varnothing$.
- The activity selection problem is to select a maximum-size set of mutually non-interfering activities for use of the resource.

Greedy Algorithms: Activity Selection

Greedy Algorithms: Activity Selection

So how do we schedule the largest number of activities on the resource?

- Intuitively, we do not like long activities
- Because they occupy the resource and keep us from honoring other requests.

Greedy Algorithms: Activity Selection

Greedy Algorithms: Activity Selection

This suggests the following greedy strategy:

- Repeatedly select the activity with the smallest duration (f_i - s_i) and schedule it, provided that it does not interfere with any previously scheduled activities.

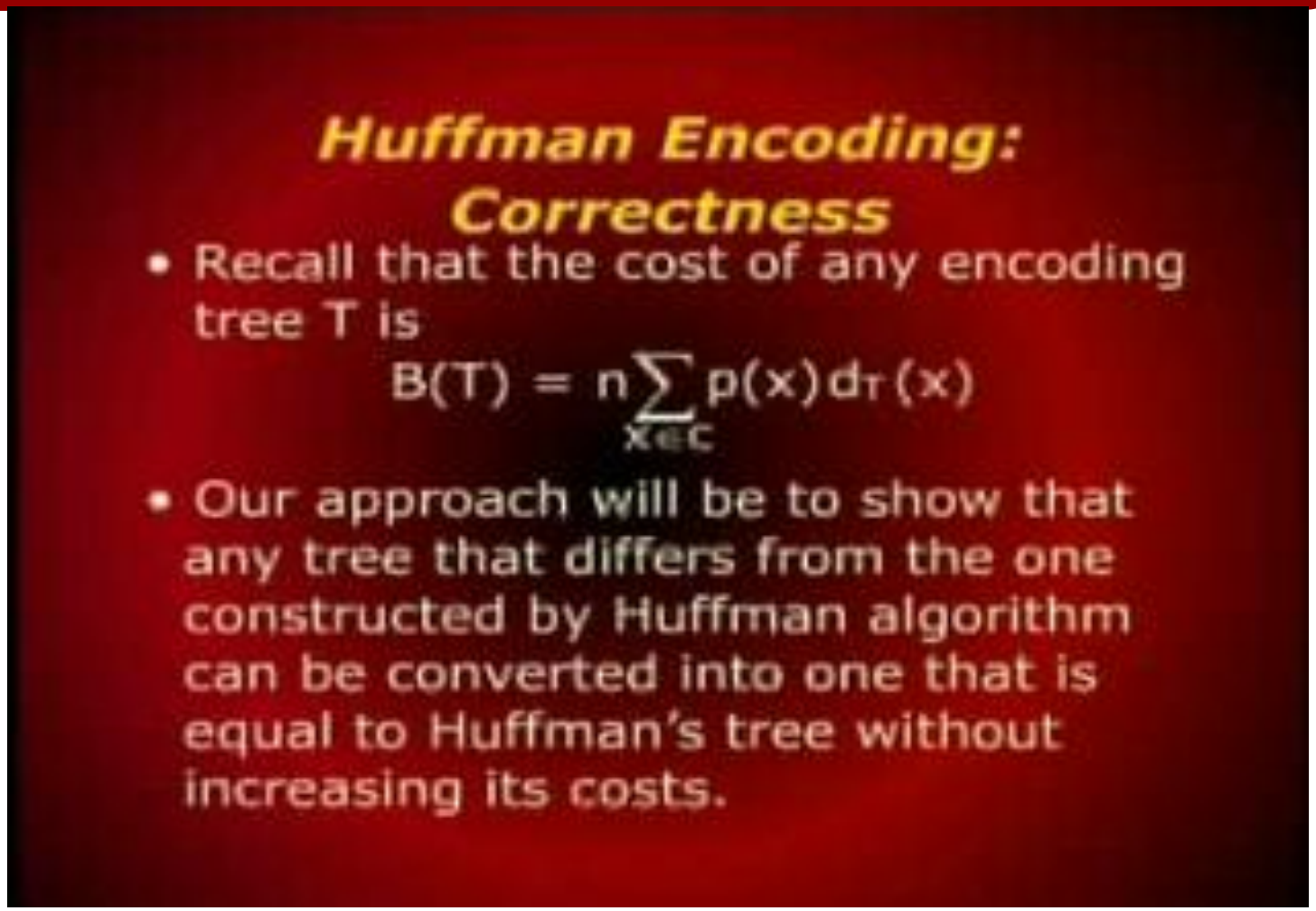
Greedy Algorithms: Activity Selection

Greedy Algorithms: Activity Selection

Here is a simple greedy algorithm that works:

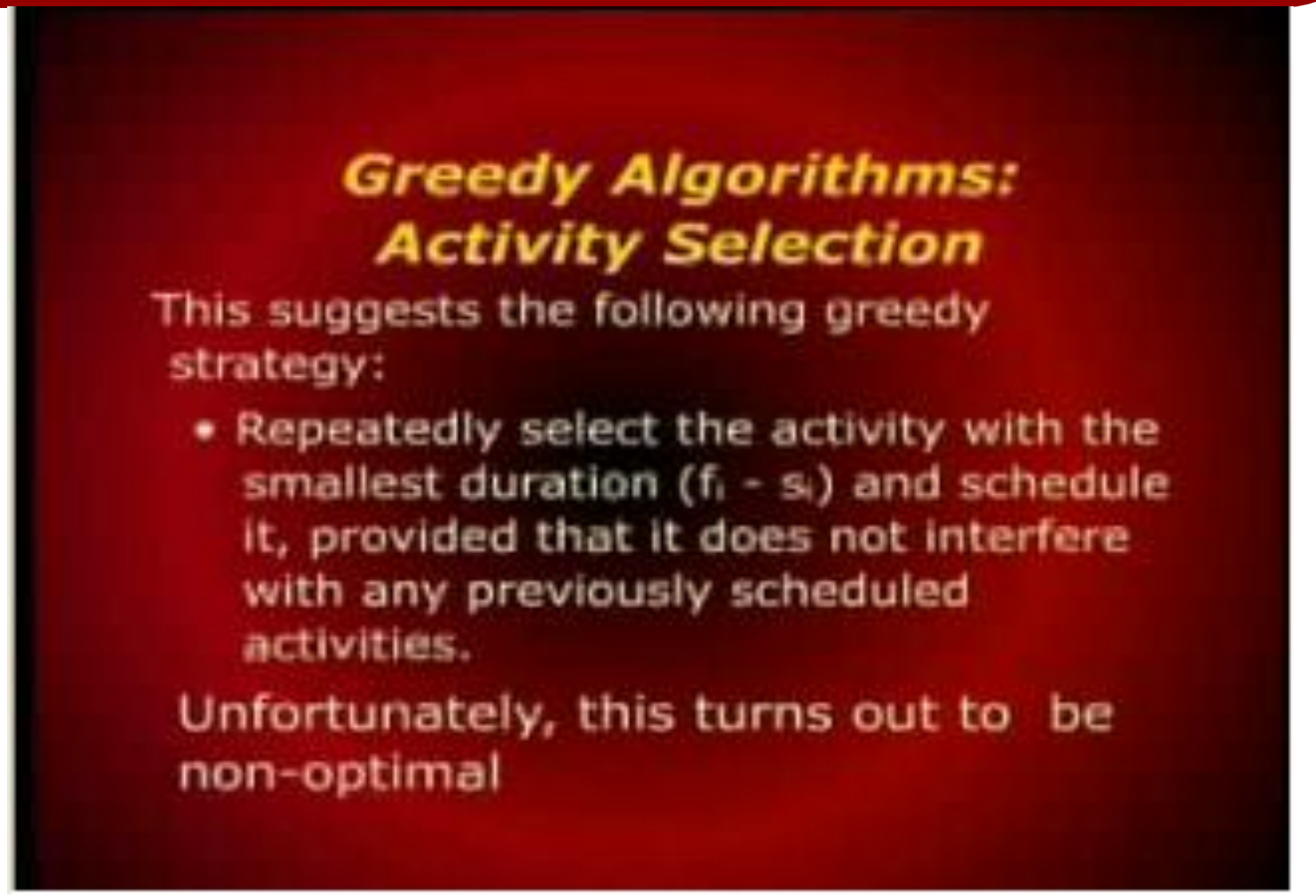
- Sort the activities by their finish times.
- Select the activity that finishes first and schedule it.
- Then, among all activities that do not interfere with this first job, schedule the one that finishes first, and so on.

Huffman Encoding: Correctness

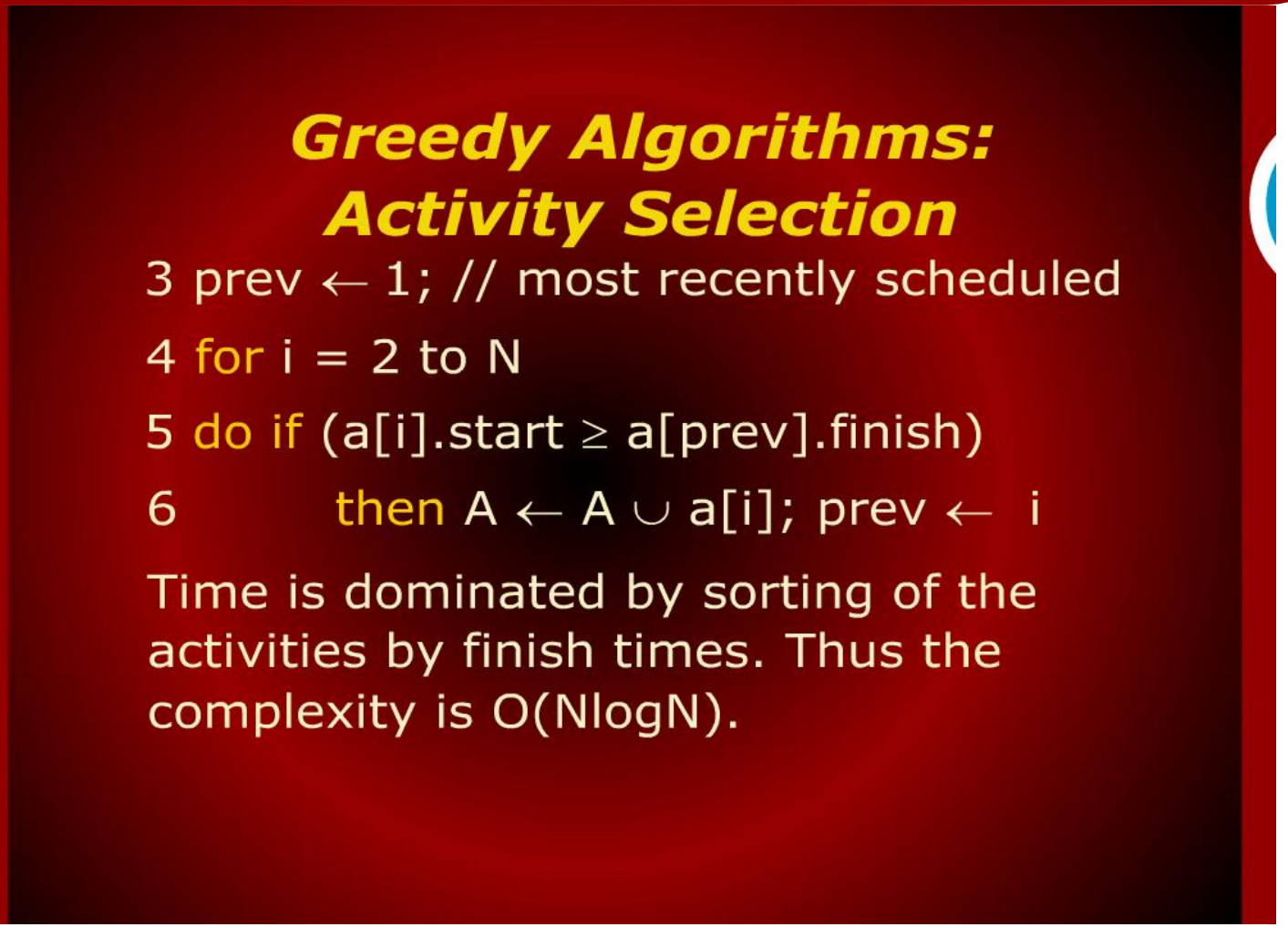


Huffman Encoding

Greedy Algorithms: Activity Selection



Greedy Algorithms: Activity Selection



Greedy Algorithms: Activity Selection